# LoRA4VR — Training Log

A living record of experiments, findings, and open questions

**What this is.** A dated log of experiments run on the LoRA4VR multi-turn RL loop. Each entry records what was tried, what happened, and how to reproduce it. Entries are added as experiments complete. Reproduction references a git commit; the code at that commit is the exact code that produced the result.

**System.** `Qwen3-4B-Instruct-2507` with a LoRA adapter trained by branched multi-turn GRPO (`src/branched_trainer.py`). The model writes `policy(state) -> action` Python for the slosh-tank environment (`scripts/seed_tasks/slosh_tank_env.py`); the policy is scored in a sandboxed subprocess. Method and experimental design: `docs/paper.tex`.

---

## 2026-10-24 Log reset: clean slate

Every entry recorded before this date was discarded, together with the environments and runs it described (CartPole and puck-on-tilted-plane training runs, Qwen2.5-Coder-3B frozen baselines, the slosh gain sweep, the slosh frozen one- and two-turn baselines, and the first branched slosh training run). None of those results is to be cited or built on. The slosh tank is the only environment going forward; the experiment ladder restarts at the environment-validity gate described in `docs/paper.tex`.

---

## 2026-10-24 Rung 0 gate: hiding the slosh sensor and the slosh length does not make the lab necessary

`b6485c0` · *hand-written controllers only, CPU; no model was run*

### Setup / motivation

The ladder restarts with a validity gate: before any model is sampled, show with hand-written controllers that the scored `policy(state)` cannot succeed without information that only the turn-1 lab can provide. Two facts about the env as shipped were established first (200 seeds per value). With the actuator gain $\kappa \in \{0.65, 0.80, 1.00\}$ hidden, a fixed PD controller with the gain constant hard-coded to 0.65 succeeds in 100% of episodes at every $\kappa$; the shipped "blind" reference is the same formula with the constant at 1.0 (0 / 50.5 / 100%), so the gap it exhibited was a tuning artifact. With the slosh length $L$ varied from 0.3 to 1.5 m and the full state observed, a PD controller that knows nothing about $L$ also succeeds in 100% of episodes at every $L$: feedback on $\theta$ does the probe's job for free.

The redesign under test therefore hid the slosh state. The scored controller observed only $(t, x, v, \text{goal}, \text{steps_left})$; $\theta$ and $\omega$ were visible only to the turn-1 lab env. The hidden parameter was $L$ (slosh period $T_n = 2\pi\sqrt{L/g}$), with $\kappa$ fixed at 1. Because $\theta$ never enters the cart update in

`SloshTankEnv.step`, the scored controller's inputs are independent of $L$, so no controller, stateful or not, can identify $L$ in-episode; the only blind strategy is robustness. The gate measured what robustness costs. A *blind* side (one controller for every $L$ in the set) and an *informed* side (one controller per $L$, the upper bound on what a perfect turn-1 measurement could buy) searched the same five controller classes with the same budget on the same 48 search seeds: shaped bang-bang moves (none/ZV/ZVD/ZVDD/EI/two-mode shapers plus a terminal PD), free-form antisymmetric open-loop profiles with 8 segments per half, pure PD on $(x, v)$, shaped PD with memory, and feedforward plus tracking feedback. Each job ran 800 random samples and $8 \times 150$ hill-climbing steps; winners were chosen on the search seeds and reported on 200 fresh seeds per $L$. The gates were fixed beforehand: G1, informed $\geq 90\%$ at every $L$; G2, informed mean minus blind mean $\geq 40$ percentage points; G3, a hand-written two-turn solution (lab measurement, then controller) $\geq 95\%$ on fresh seeds. Twelve configurations were run (1.3 CPU-hours, 7 minutes on 11 cores).

**Results**

| $L$ set (m) | steps | blind per $L$ (%) | blind mean | informed per $L$ (%) | informed mean | gap (pp) |
|---|---|---|---|---|---|---|
| $\{2, 3, 4\}$ | 80 | 100 / 100 / 0 | 67 | 100 / 100 / 99 | 100 | +33 |
| $\{2, 3, 4\}$ | 90 | 96 / 100 / 86 | 94 | 99 / 100 / 100 | 100 | +5 |
| $\{2, 3, 4\}$ | 105 | 100 / 100 / 100 | 100 | 99 / 100 / 100 | 100 | 0 |
| $\{3, 4, 5\}$ | 90 | 98 / 98 / 98 | 98 | 100 / 100 / 100 | 100 | +2 |
| $\{3, 4, 5\}$ | 105, 120, 135 | 100 / 100 / 100 | 100 | 100 / 100 / 100 | 100 | 0 |
| $\{2.5, 3.5, 4.5\}$ | 105 | 99 / 100 / 100 | 100 | 100 / 100 / 100 | 100 | 0 |
| $\{2.5, 3.5, 4.5\}$ | 120 | 100 / 100 / 100 | 100 | 100 / 100 / 100 | 100 | 0 |
| $U[2, 4]$ | 90 | 100 / 100 / 100 | 100 | 100 / 100 / 100 | 100 | 0 |
| $U[3, 5]$ | 105 | 100 / 100 / 82 | 94 | 100 / 100 / 100 | 100 | +6 |
| $U[3, 5]$ | 120 | 96 / 100 / 100 | 99 | 100 / 100 / 100 | 100 | +1 |

G1 passed everywhere. G2 failed in every configuration. The blind winner was always a free-form open-loop profile, typically a short bang-bang convolved with a two-frequency shaper, whose success on the continuous ranges was 100% over uniformly drawn $L$. G3 passed at 100% for every $L$ in every configuration with at least 90 steps: the lab code applied a four-step pulse and read the period off the zero crossings of $\theta$ to within 0.2% (e.g. 4.008 s measured against 4.012 s true), chose the move acceleration on lab goals disjoint from the scoring goals, and the turn-2 controller was a ZV-shaped bang-bang with the two constants filled in. At 80 steps G3 fell to 90.5% at $L = 4$.

**Interpretation**

The premise of the redesign holds, but an unobservable parameter is not the same as a necessary measurement. Robust input shaping over $\pm 20\%$ of frequency uncertainty costs little time, and the optimizer found profiles that keep the residual slosh inside the tolerance for all three periods at once. The +58 pp gap seen in the pilot came from a controller family that contained only classical single-frequency shapers; once the blind side was allowed free-form profiles it vanished. A gap appears only when the budget is squeezed below what the longest period needs, and there it also hurts the informed side (G3 at 80 steps), so it is a budget artifact rather than an information gap.

Three constraints on any future design follow. First, anything the scored controller observes that depends on the hidden quantity reveals it within a step or two, because the dynamics are deterministic; the hidden quantity must affect only variables the scored controller cannot see. Second, if the hidden quantity admits a robust compromise, the lab buys nothing; frequency uncertainty of this width admits one. Third, a scale-like parameter such as the actuator gain

admits no compromise under a fixed horizon, but the env's termination on first success lets a blind open-loop plan test hypotheses in sequence: with $\kappa \in \{0.65, 0.8, 1.0\}$ a plan that stops at planned positions $d$, $1.25d$ and $1.54d$ succeeds under each value in turn. Success would have to be judged at the final step.

### Hypotheses / next steps

1. Run the same gate on a dead-reckoning variant: the scored controller observes only $(t, \text{goal}, \text{steps_left})$, the gain (and optionally the period) is hidden, and success is evaluated at the last step with no early termination. The blind side can then be right for at most one gain value, so the gap exists by construction; the gate would confirm the informed side reaches $\geq 90\%$ and the hand-written two-turn solution $\geq 95\%$.
2. If it passes, push the two-turn solution through the real runner with the observation filter, a lab proxy that hides `env.hidden`, a lab seed disjoint from the scoring seeds, and scoring on at least four goals.
3. Check at Rung 2 that a frozen model does not already solve the lab step, since measuring a gain takes one line of code.

### Open questions

Whether the no-sensor story is physically acceptable to a reviewer; whether judging success at the final step sparsifies the reward enough to hurt single-turn RL at Rung 3; whether slosh is still worth keeping once the gain carries the information gap.

### Reproduce

`b6485c0`, `scripts/rung0_gate.py`: `search --n-random 800 --topk 8 --n-climb 150` over the twelve configurations listed above, and `two-turn --L 2 3 4 --steps 90`. Raw output: `docs/rung0/search_2026-10-24.log`.